



Trabajo Práctico Integrador Programación

Gestión de Datos de Países en Python

Alumno: Matías Fiorentini
Comisión N° 13

Marco teórico	2
Listas	3
Diccionarios	3
Funciones	4
Estructuras condicionales	4
Ordenamientos	5
Estadísticas	5
Manejo de archivos CSV	6
Decisiones técnicas y arquitectura	7
Arquitectura general	7
Diagrama de flujo de las operaciones principales	8
Capturas de pantalla de la ejecución	9
Dificultades y conclusiones	14
Links al repositorio y al video	15

◆ Marco teórico

• Listas

Una lista es una secuencia de elementos guardados uno detrás del otro, a la que se le puede ir agregando o sacando elementos mientras el programa está corriendo, sin necesidad de crear una variable nueva cada vez. Es la estructura que elegí para representar todo el dataset: cada país ocupa un lugar dentro de una lista, y esa lista completa es el conjunto de datos sobre el que trabaja todo el programa.

```
lista_paises = [
    {"nombre": "Argentina", "poblacion": 45376763, "superficie":
    2780400, "continente": "América"},
    {"nombre": "Brasil", "poblacion": 213993437, "superficie":
    8515767, "continente": "América"},
]
```

Como cada elemento de la lista es, a su vez, un diccionario, lista_paises es lo que se conoce como una lista de diccionarios: la lista es el contenedor que se puede modificar y recorrer, y cada diccionario adentro representa los datos de un país. Este comportamiento de las listas está descripto en la documentación oficial de Python.

Fuentes:

- <https://docs.python.org/es/3.14/library/stdtypes.html#lists>
- https://www.w3schools.com/python/python_lists.asp

• Diccionarios

Un diccionario, a diferencia de la lista, no numera sus elementos, sino que les pone un nombre (la clave) a cada uno. En vez de tener que acordarme que la población está en la posición 1 y la superficie en la posición 2, puedo simplemente pedir pais["poblacion"] o pais["superficie"], es decir, se puede hacer referencia a ellos utilizando el nombre de la clave. Por eso cada país del proyecto se guarda como un diccionario con cuatro claves fijas, en lugar de, por ejemplo, una tupla de valores sueltos.

```
pais = {"nombre": "Japón", "poblacion": 125800000, "superficie":
377975, "continente": "Asia"}
print(pais["poblacion"])
# 125800000
```

Fuentes:

- https://www.w3schools.com/python/python_dictionaries.asp
- <https://docs.python.org/es/3.7/tutorial/datastructures.html#dictionaries>

- **Funciones**

Una función te permite definir un bloque de código reutilizable que se puede ejecutar muchas veces dentro de tu programa. Con respecto al programa Gestión de Países, en vez de escribir todo el programa de corrido, lo dividí en muchas funciones chiquitas (una para validar un dato, otra para guardar el archivo, otra para buscar un país), cada una responsable de una sola cosa. Esto hizo que, cuando algo no funcionaba como esperaba, fuera mucho más fácil encontrar en qué parte puntual del código estaba el problema.

```
def obtener_poblacion(pais):  
    return pais["poblacion"]
```

Fuentes:

- <https://www.freecodecamp.org/espanol/news/guia-de-funciones-de-python-con-ejemplos/>
- https://www.w3schools.com/python/python_functions.asp

- **Estructuras condicionales**

En Python, las operaciones condicionales permiten controlar el flujo de tu programa. El bloque clásico if evalúa expresiones booleanas, mientras que match evalúa patrones y es ideal para comparar un valor contra múltiples opciones

Además del clásico if/elif/else, en el programa usé match/case, que es la versión moderna del switch de otros lenguajes: compara el número que tipeó el usuario contra cada opción posible del menú, y ejecuta solo el bloque que corresponde.

```
match opcion:  
    case "1":  
        agregar_pais(lista_paises)  
    case "0":  
        break
```

Fuente:

- <https://medium.com/@sysglobalsolutionsblog/condicionales-en-python-if-else-y-match-78490a8eb304>

• Ordenamientos

Ordenar una lista significa reorganizar sus elementos según un criterio de comparación determinado, de manera que queden dispuestos en una secuencia particular (alfabética, de menor a mayor, etc.) en lugar del orden en que fueron ingresados originalmente.

Para ordenar la lista de países no escribí ningún algoritmo de ordenamiento a mano: Python ya trae la función `sorted()`, que devuelve una lista nueva ordenada sin tocar la original. El detalle es que `sorted()` no sabe por sí sola si tiene que comparar por nombre, población o superficie, así que hay que decírselo con el parámetro `key`, pasándole una función que indique qué valor mirar de cada país. El parámetro `reverse`, por su parte, es el que permite elegir entre orden ascendente o descendente.

```
sorted(lista_paises, key=obtener_poblacion, reverse=True)
```

Fuentes:

- <https://developers.google.com/edu/python/sorting?hl=es-419>
- <https://www.freecodecamp.org/news/python-sort-dictionary-by-key/>

• Estadísticas

Para las estadísticas no hizo falta programar nada demasiado elaborado: usé funciones que Python ya trae incorporadas. `max()` y `min()` (con el mismo parámetro `key` que `sorted()`) para encontrar el país con mayor y menor población; `sum()` y `len()` para calcular promedios, dividiendo la suma total entre la cantidad de países; y el método `.get()` de los diccionarios para ir contando cuántos países hay por cada continente, sin que el programa se rompa la primera vez que aparece un continente nuevo.

```
total = sum(pais["poblacion"] for pais in lista_paises)
promedio = total / len(lista_paises)
```

Fuentes:

- <https://indhumathychelliah.com/2020/08/17/how-to-use-key-function-in-max-and-min-in-python/>
- <https://www.freecodecamp.org/espanol/news/guia-de-funciones-de-python-con-ejemplos/>

- **Manejo de archivos CSV**

Un archivo CSV es un archivo de texto donde cada línea es un registro y los datos van separados por comas.

En relación al programa lo que hice fue abrir el archivo con `open()` dentro de un bloque `with` (así se cierra solo, incluso si algo sale mal en el medio), y a cada línea le sacó el salto de línea con `.strip()` y la separé en sus cuatro partes con `.split(",")`. Para escribir hice el camino inverso, uniendo los cuatro datos con `",".join()`. Si una línea viene mal escrita o el archivo no existe, el programa avisa y sigue funcionando en lugar de romperse, gracias a manejar esos casos puntuales con `try/except`.

```
linea = "Argentina,45376763,2780400,América"
campos = linea.strip().split(",")
# campos -> ['Argentina', '45376763', '2780400', 'América']
```

Fuentes:

- <https://www.freecodecamp.org/espanol/news/manejo-de-archivos-en-python-como-crear-leer-y-escribir-en-un-archivo/>
- <https://mimo.org/glossary/python/file-handling>

◆ Decisiones técnicas y arquitectura

- **Arquitectura general**

El programa se desarrolló como un único módulo de Python (`gestion_Paises.py`), organizado internamente en secciones claramente comentadas: validaciones, manejo de archivos, gestión de altas y búsquedas, filtros, funciones auxiliares para ordenamiento, ordenamientos, estadísticas, presentación por consola, y el menú con el control principal.

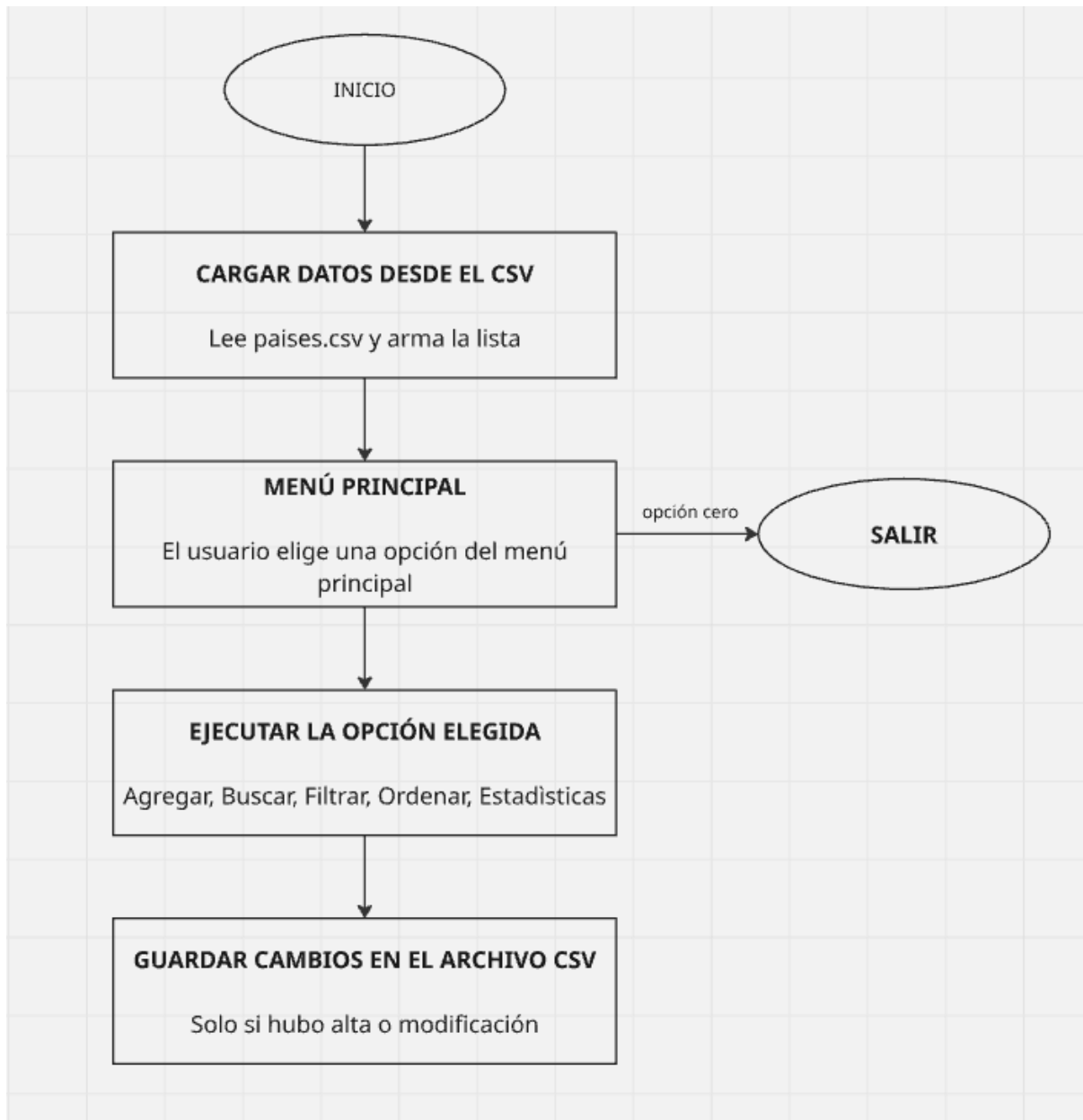
Toda la información se mantiene en memoria como una lista de diccionarios (`lista_paises`) durante la ejecución, y se sincroniza con el archivo `paises.csv` únicamente cuando hay un cambio real en los datos (alta o actualización); las operaciones de consulta (buscar, filtrar, ordenar, estadísticas) son de solo lectura y no vuelven a escribir el archivo.

Algunas decisiones de diseño tomadas durante el desarrollo:

- Se separó la funcionalidad del programa en diferentes funciones para que cada una se encargue de una sola tarea puntual, facilitando probar, leer y corregir cada parte del código de forma independiente.
- Se usó `match/case` en lugar de `if/elif` para el menú principal y los submenús, aprovechando que es la forma estándar de Python para este tipo de decisión múltiple,
- Se investigó el uso del parámetro `key` en `sorted()`, `max()` y `min()`, ya que estas funciones no tienen una forma directa de comparar diccionarios entre sí: necesitan que se les indique qué valor de cada país usar como criterio de comparación. Para eso se definieron funciones con `def` (`obtener_poblacion`, `obtener_superficie`, `obtener_nombre_minuscula`), cada una encargada de devolver el valor puntual por el que se quería comparar, de manera que quedara claro y reutilizable qué criterio se le estaba pasando a cada `sorted()`, `max()` o `min()`.
- Los filtros se implementaron recorriendo la lista completa de países con un bucle `for`, y usando `.append()` para ir agregando a una lista nueva (`resultado`) únicamente los países que cumplen la condición pedida (el continente, el rango de población o el rango de superficie). Es el patrón de recorrido con acumulador: se arranca con una lista vacía y se la va completando a medida que se encuentran elementos que corresponden.
- El menú se organizó en dos niveles (menú principal y submenús de Filtrar y Ordenar), en vez de un único menú plano, para que cada bloque de opciones quedara corto y fácil de leer.

- **Diagrama de flujo de las operaciones principales**

El siguiente diagrama resume el ciclo de vida del programa: la carga inicial del CSV, el bucle del menú principal, la ejecución de la opción elegida, el guardado de cambios en el CSV, y la salida del programa.



- Capturas de pantalla de la ejecución

- Menù principal

```
===== GESTIÓN DE PAÍSES =====  
1. Agregar país  
2. Actualizar población y superficie  
3. Buscar país por nombre  
4. Filtrar países  
5. Ordenar países  
6. Mostrar estadísticas  
0. Salir  
=====
```

Seleccione una opción:

- Agregar País

```
=====
```

Seleccione una opción: 1

```
--- Agregar nuevo país ---  
Nombre del país: Chile  
Población: 20200000  
Superficie en km²: 756000  
Continente: America  
País 'Chile' agregado correctamente.
```

- Actualizar población y superficie

```
Seleccione una opción: 2
```

```
--- Actualizar población y superficie ---  
Nombre del país a actualizar: Argentina  
  Datos actuales -> población: 51000000, superficie: 2780400 km²  
Nueva población: 52000000  
Nueva superficie en km²: 2781400  
País 'Argentina' actualizado correctamente.
```

- Buscar país por nombre

```
Seleccione una opción: 3
Nombre (o parte del nombre) a buscar: Argentina

Se encontraron 1 país/es:
Nombre | Población | Superficie (km2) | Continente
-----
Argentina | 51000000 | 2780400 | America
```

- Filtrar países

```
Seleccione una opción: 4

--- Filtrar países ---
1. Por continente
2. Por rango de población
3. Por rango de superficie
Seleccione una opción: 1
Continente: America

Se encontraron 6 país/es:
Nombre | Población | Superficie (km2) | Continente
-----
Argentina | 51000000 | 2780400 | America
Brasil | 213993437 | 8515767 | America
México | 128972439 | 1964375 | America
Estados Unidos | 331893745 | 9833517 | America
Peru | 30000000 | 3000000 | America
Chile | 20200000 | 756000 | America
```

```

Seleccione una opción: 4

--- Filtrar países ---
1. Por continente
2. Por rango de población
3. Por rango de superficie
Seleccione una opción: 2
Población mínima: 10000000
Población máxima: 52000000

Se encontraron 4 país/es:
Nombre | Población | Superficie (km2) | Continente
-----
Argentina | 51000000 | 2780400 | America
Australia | 25788215 | 7692024 | Oceanía
Peru | 30000000 | 3000000 | America
Chile | 20200000 | 756000 | America

```

```

Seleccione una opción: 4

--- Filtrar países ---
1. Por continente
2. Por rango de población
3. Por rango de superficie
Seleccione una opción: 3
Superficie mínima (km²): 2000000
Superficie máxima (km²): 15000000

Se encontraron 5 país/es:
Nombre | Población | Superficie (km2) | Continente
-----
Argentina | 51000000 | 2780400 | America
Brasil | 213993437 | 8515767 | America
Estados Unidos | 331893745 | 9833517 | America
Australia | 25788215 | 7692024 | Oceanía
Peru | 30000000 | 3000000 | America

```

- Ordenar países

```

-----
Seleccione una opción: 5

--- Ordenar países ---
1. Por nombre
2. Por población
3. Por superficie
Seleccione una opción: 1
¿Ascendente o descendente? (A/D): a
Nombre | Población | Superficie (km2) | Continente
-----
Alemania | 83149300 | 357022 | Europa
Argentina | 51000000 | 2780400 | America
Australia | 25788215 | 7692024 | Oceanía
Brasil | 213993437 | 8515767 | America
Chile | 20200000 | 756000 | America
Estados Unidos | 331893745 | 9833517 | America
Francia | 67391582 | 643801 | Europa
Japón | 125800000 | 377975 | Asia
México | 128972439 | 1964375 | America
Peru | 30000000 | 3000000 | America

```

```

Seleccione una opción: 5

--- Ordenar países ---
1. Por nombre
2. Por población
3. Por superficie
Seleccione una opción: 2
¿Ascendente o descendente? (A/D): a
Nombre | Población | Superficie (km2) | Continente
-----
Chile | 20200000 | 756000 | America
Australia | 25788215 | 7692024 | Oceanía
Peru | 30000000 | 3000000 | America
Argentina | 51000000 | 2780400 | America
Francia | 67391582 | 643801 | Europa
Alemania | 83149300 | 357022 | Europa
Japón | 125800000 | 377975 | Asia
México | 128972439 | 1964375 | America
Brasil | 213993437 | 8515767 | America
Estados Unidos | 331893745 | 9833517 | America

```

```

Seleccione una opción: 5

--- Ordenar países ---
1. Por nombre
2. Por población
3. Por superficie
Seleccione una opción: 3
¿Ascendente o descendente? (A/D): a
Nombre | Población | Superficie (km2) | Continente
-----
Alemania | 83149300 | 357022 | Europa
Japón | 125800000 | 377975 | Asia
Francia | 67391582 | 643801 | Europa
Chile | 20200000 | 756000 | America
México | 128972439 | 1964375 | America
Argentina | 51000000 | 2780400 | America
Peru | 30000000 | 3000000 | America
Australia | 25788215 | 7692024 | Oceanía
Brasil | 213993437 | 8515767 | America
Estados Unidos | 331893745 | 9833517 | America

```

○ Estadísticas países

```

Seleccione una opción: 6

--- Estadísticas ---
País con mayor población: Estados Unidos (331,893,745)
País con menor población: Chile (20,200,000)
Promedio de población: 107,818,871.80
Promedio de superficie: 3,592,088.10 km²
Cantidad de países por continente:
- America: 6
- Asia: 1
- Europa: 2
- Oceanía: 1

```

◆ Dificultades y conclusiones

Durante el desarrollo me encontré con algunos obstáculos. El primero fue decidir cómo manejar errores de formato dentro del archivo CSV. En una primera versión, el programa fallaba por completo si una sola línea tenía un dato no numérico o un campo faltante. Lo resolví agregando un control de excepciones específico (`ValueError`) por cada línea, de modo que el programa descartara únicamente la línea problemática y continuara cargando el resto del archivo, informando el inconveniente por consola.

Otro desafío fue encontrar la forma más clara de indicarle a `sorted()`, `max()` y `min()` qué valor de cada país usar para comparar. Investigué el parámetro `key` y opté por definir funciones auxiliares con `def` (`obtener_poblacion`, `obtener_superficie`, `obtener_nombre_minuscula`), lo que mejoró la legibilidad del código al darle un nombre explícito a cada criterio de comparación.

Por otro lado, los conceptos que más afiancé con este trabajo fue el uso de listas y diccionarios como estructura central del programa: una lista de diccionarios resulta natural para representar un conjunto de registros con varios campos, como en este caso los países, y permite recorrer, filtrar y ordenar el dataset completo con pocas líneas de código. También me preocupé en dividir el programa en funciones pequeñas, cada una con una sola responsabilidad, lo que en la práctica significó poder probar cada funcionalidad (agregar, buscar, filtrar, ordenar, calcular estadísticas) de forma aislada, y encontrar errores mucho más rápido que si todo el código estuviera junto.

Otro de los conceptos aplicados fue el uso o manejo de excepciones, ya que estas me parecen importantes porque son la base de cualquier programa que necesite manejar datos reales: ningún sistema trabaja con datos perfectos, así que el manejo de excepciones (`try/except`) se vuelve una herramienta concreta para que el programa siga funcionando ante un archivo incompleto o mal escrito.

Por último, como este trabajo lo realicé de forma individual, organicé el desarrollo en etapas: primero definí la estructura de datos y el diseño de las funciones, después fui implementando y probando cada bloque por separado (lectura y escritura de archivos, gestión de altas, filtros, ordenamientos, estadísticas) antes de integrarlo al menú principal, y dejé la documentación para el final, una vez que el programa funcionaba adecuadamente.

◆ Links al repositorio y al video

Repositorio del proyecto en GitHub: <https://github.com/MatiasFiorentini/TPI-Programacion>

Video demostrativo: <https://www.youtube.com/watch?v=jgGCRCG8U00>